Fingertip Trajectory Recognition

1. Experiment Objective

Use MediaPipe Hands to track the index fingertip's movement trajectory in the air, control trajectory recording and recognition with hand gestures, and automatically classify the shape as triangle, square, circle or star.

2. Experiment Principle

1. Key Terms

Keyword	Description
Fingertip trajectory	The sequence of coordinates of the index fingertip (landmark 8) over time, recording the air-writing path
Trajectory classification	Automatically classifies the trajectory into a geometric shape based on shape features (number of corners, aspect ratio, circularity, etc.)
Record/recognize gestures	`one` (index finger extended, other four fingers bent): hold it for several consecutive frames to trigger recording; `five` (all five fingers spread): ends recording and triggers automatic classification. Gestures are recognized from finger joint angles, not a simple y-coordinate comparison

2. Technical Architecture

```
Terminal 1: start agent + camera + joint models
Terminal 2: start hand detection
```

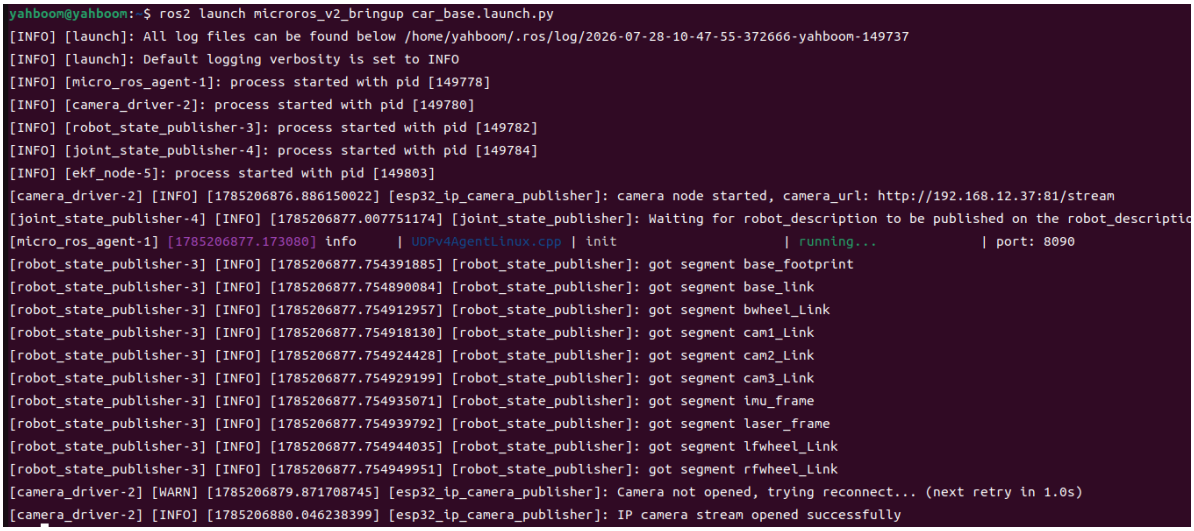
3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☑ Supported
No-PTZ version	☑ Supported
Required peripherals	ESP32 camera

2. Start the agent + camera + joint models (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```



3. Start fingertip trajectory recognition (Terminal 2)

```
ros2 launch microros_v2_mediapipe_pkg finger_trajectory.launch.py
```

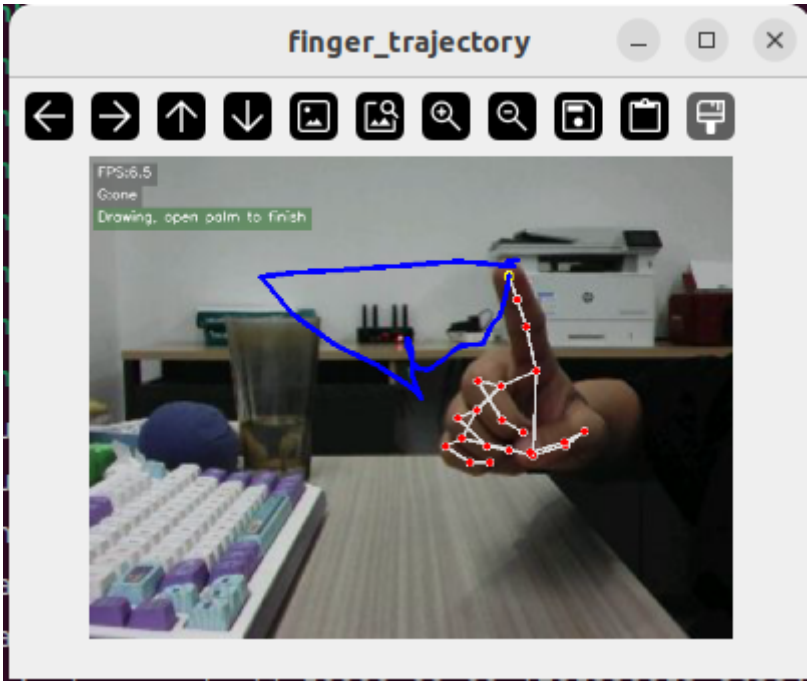
4. Operating Instructions

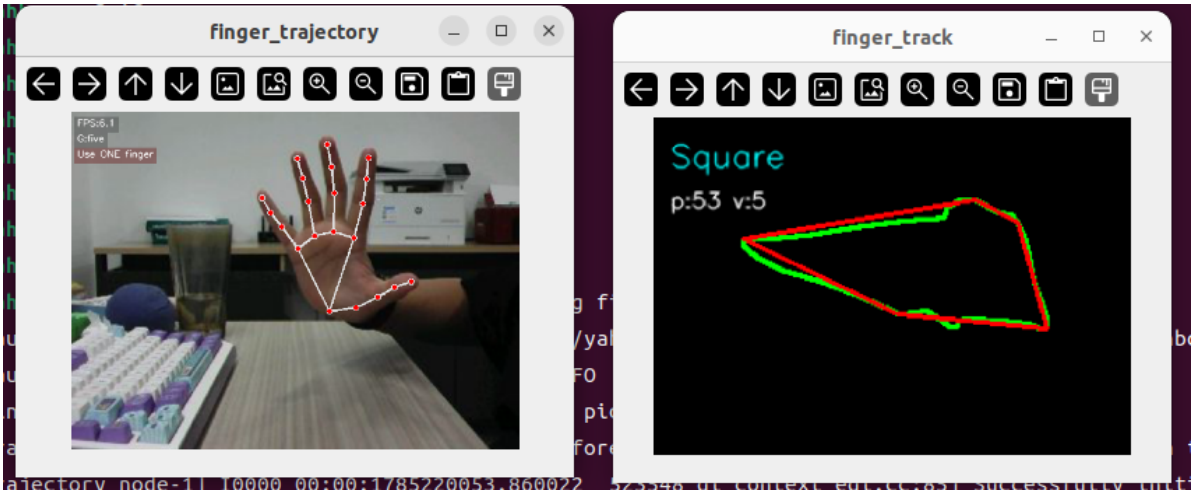
Step	Gesture	Description
Start recording	`one` (index finger extended, other four fingers bent)	Hold it for about several consecutive frames (5 frames); the progress bar on screen confirms, and the prompt `START DRAWING` appears
Draw the trajectory	Keep the index finger extended	Draw a geometric shape (triangle/square/circle/star) in the air with the fingertip; the trajectory is drawn in real time with red lines
Finish and recognize	`five` (all five fingers spread)	Recording ends immediately; contour extraction → polygon approximation → corner classification runs automatically, and the result shows `Result: Triangle/Square/Circle/Star`
Cancel / redo	Press the space bar	Clear the current trajectory points and draw again
Exit	Press `q`	Exit the program

> Gestures are recognized via `h_gesture(hand_angle(landmarks))`: the flexion angles of the five fingers' joints are computed and compared against thresholds to classify the gesture as `one` / `five` / `none` — not a simple comparison of fingertip y-coordinates.

4. Observed Results

- As the index fingertip moves across the frame, the trajectory is drawn in real time with colored lines
- After recording finishes, the trajectory is automatically classified as triangle/square/circle/star
- The recognition result is shown in the on-screen overlay info area





5. Code Walkthrough

> Source code paths:
>
> - Launch: `~/microros_ws/src/microros_v2_mediapipe_pkg/launch/finger_trajectory.launch.py`
> - Node: `~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/FingerTrajectory.py`

1. Core Node Analysis

`FingerTrajectory` recognizes gestures from finger joint angles and uses a state machine to control trajectory recording and shape classification.

Gesture recognition (`h_gesture`) — based on finger joint angles

```
# Source file:
~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/FingerTrajectory.py
def h_gesture(angle_list):
    """
```

```
    five: all five fingers spread, used to end the trajectory
    one : index finger extended, used to prepare and draw the trajectory
    none: other gestures
    """
    thr_angle = 65.0
    thr_angle_s = 49.0

    if all(a < thr_angle_s for a in angle_list):
        return "five"

    if (
        (angle_list[0] > 5)
        and (angle_list[1] < thr_angle_s)
        and (angle_list[2] > thr_angle)
        and (angle_list[3] > thr_angle)
        and (angle_list[4] > thr_angle)
    ):
        return "one"

    return "none"
```

Key logic:

- `hand_angle(landmarks)`: computes the joint flexion angles of the 5 fingers (thumb[0], index[1], middle[2], ring[3], pinky[4]), using `vector_2d_angle` to compute the 2D vector angles between adjacent phalanges
- `five` **(all fingers spread)**: all 5 angles < 49°, fingers fully extended → **ends recording and triggers classification**
- `one` **(index extended)**: thumb > 5° (not pressed against the palm) + index < 49° (extended) + middle/ring/pinky > 65° (bent), i.e. only the index finger is extended
- `none`: any other gesture, ignored

State machine (`get_rgb_image_callback`) — gesture-driven recording control

```
# Source file:
~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/FingerTraj
ectory.py
class FingerTrajectory(Node):
    def __init__(self):
        self.state = State.NULL          # idle / recording
        self.points = []                 # trajectory point list
        self.start_count = 0             # "one" consecutive frame count
        self.start_count_threshold = 5   # requires 5 consecutive frames to
confirm (prevents mis-triggering)

    def get_rgb_image_callback(self, msg):
        # ... decode image, MediaPipe Hands inference ...

        if self.state == State.NULL:
            if gesture == "one":
                self.start_count += 1
                if self.start_count >= self.start_count_threshold:
                    self.start_tracking()  # enter the recording state
            else:
                self.start_count = 0       # gesture interrupted, reset the
count
```

```
        elif self.state == State.TRACKING:
            if gesture == "five":
                self.finish_tracking()     # finish recording → shape
classification
            elif index_finger_tip is not None:
                tip = self.smooth_point(index_finger_tip)
                # minimum distance filtering to avoid duplicate sampling when
stationary
                if len(self.points) == 0 or distance(self.points[-1], tip) >=
self.min_point_distance:
                    self.points.append(tip)
```

Key logic:

- **Debounce mechanism**: `one` must be held for `start_count_threshold=5` consecutive frames before recording triggers, preventing transient mis-triggers during gesture transitions
- **Lost timeout**: if no hand is detected for longer than `lost_timeout=3.0s` during recording, recording is canceled automatically
- **Point-distance filtering**: `min_point_distance=5.0px`, only sampling new points at least 5px apart to avoid redundant trajectory points from stationary jitter
- **Exponential smoothing**: `smooth_alpha=0.55` applies EMA smoothing to fingertip coordinates, reducing noise from slight hand tremor

Shape classification (`finish_tracking`) — contour → polygon approximation → corner counting

```
# Source file:
~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/FingerTraj
ectory.py
def finish_tracking(self):
    # 1. trajectory points → binary image (normalized to the minimum bounding box
+ 100px margin)
    track_img = get_track_img(self.points)

    # 2. contour extraction + morphological filtering (erode → dilate,
morph_iterations=2)
    contour = get_area_max_contour(contours, self.contour_area_threshold)

    # 3. Douglas-Peucker polygon approximation (epsilon = 0.026 × perimeter)
    epsilon = self.approx_epsilon_ratio * cv.arcLength(contour, True)
    approx = cv.approxPolyDP(contour, epsilon, True)

    # 4. corner count → shape name
    if len(approx) == 3:          graph_name = "Triangle"
    elif len(approx) in (4, 5):   graph_name = "Square"
    elif 5 < len(approx) < 10:    graph_name = "Circle"
    elif len(approx) == 10:       graph_name = "Star"
```

Key logic:

- `approxPolyDP(epsilon=0.026*perimeter)`: uses 2.6% of the contour perimeter as the approximation precision, balancing polygon simplification against preserving shape features
- Corner segmentation mapping: 3=triangle, 4~5=square (rounded corners may produce 5 vertices), 6~9=circle, 10=star

- Morphological filtering: the `erode → dilate` closing operation fills small breaks in the trajectory, preventing contour fragmentation from causing misclassification
- Area threshold: `contour_area_threshold=200` filters out tiny false contours caused by hand tremor

2. Key Parameters

> Parameter definition files:
>
> - Launch: `~/microros_ws/src/microros_v2_mediapipe_pkg/launch/finger_trajectory.launch.py`
> - Node: `~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/FingerTrajectory.py`

Parameter	Type	Default	Description
`min_detection_confidence`	float	0.5	Detection confidence threshold
`min_tracking_confidence`	float	0.5	Tracking confidence threshold

3. Node Description

Node	Function
`finger_trajectory_node`	MediaPipe Hands index fingertip trajectory acquisition + finger state determination + shape classification

6. Frequently Asked Questions

Issue	Cause	Solution
Trajectory recording does not trigger	Finger state determination is inaccurate	Make sure only the index finger is extended and the other fingers are bent
Shape classification errors	The trajectory is not clean enough	Try to draw standard geometric shapes and avoid overlapping